GUIA DE ESTUDOS · QUARKUS REST

Do CRUD simples à segurança com JWT

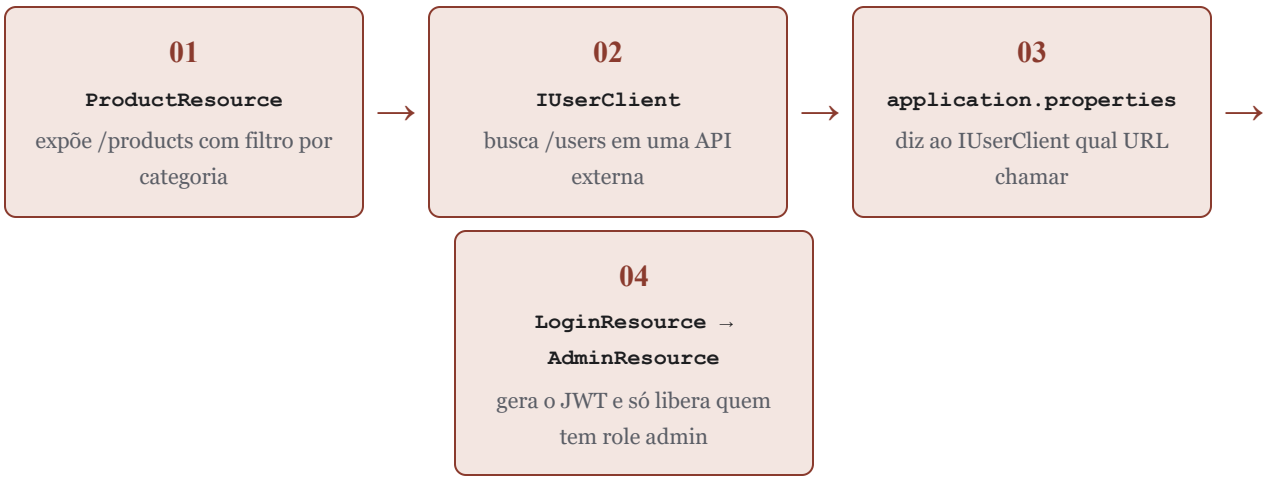
Passo a passo teórico + prático: recurso REST com filtro por query param, REST Client para consumir API externa, configuração de endpoint no estilo MicroProfile e proteção de rota por role usando JSON Web Token.

modelo didático baseado em pw2.rpmhub.dev

SUMÁRIO

1. Exercício 1 — Product e ProductResource (Web Services)
2. Exercício 2 — REST Client (IUserClient)
3. Exercício 3 — Configuration: mp-rest/url
4. Exercício 4 — Segurança com JWT e @RolesAllowed

COMO OS 4 EXERCÍCIOS SE ENCAIXAM



01 Product e ProductResource 🌐

Tema: Web Services — expor dados via HTTP/JSON usando JAX-RS.

O que é um Web Service REST? É uma forma de dois sistemas conversarem pela internet usando o protocolo HTTP. Em vez de trocar mensagens em XML complexo (como no antigo padrão SOAP/WSDL), o REST usa os próprios verbos HTTP — `GET`, `POST`, `PUT`, `DELETE` — para representar ações sobre **recursos** (no nosso caso, produtos).

No Quarkus, cada recurso é uma classe Java anotada com `@Path`. O framework (JAX-RS / RESTEasy Reactive) cuida de toda a conversão HTTP ⇄ Java ⇄ JSON — você só escreve a lógica de negócio.

PASSO 1 — MODELAR O PRODUTO COMO RECORD

Como o `Product` só carrega dados (não tem comportamento próprio), o jeito moderno e idiomático em Java é usar um **record**: uma classe imutável onde o compilador gera automaticamente construtor, getters (`id()`, `name()`, `category()`), `equals()`, `hashCode()` e `toString()`. É exatamente o padrão que a apostila usa para o `Produto`.

```
package org.acme;

public record Product(Long id, String name, String category) {
}
```

Por que record e não classe com campos públicos? Menos código repetido, imutabilidade por padrão (mais seguro) e o Jackson (motor de JSON do Quarkus) já sabe serializar/desserializar records automaticamente.

● PASSO 2 — CRIAR O RECURSO REST

`@Path("/products")` na classe define a rota base. O método marcado com `@GET` responde a requisições HTTP GET; `@Produces(MediaType.APPLICATION_JSON)` avisa o cliente que a resposta vem em JSON. O parâmetro `@QueryParam("category")` captura o valor depois do `?category=` na URL — é opcional por natureza (chega `null` se não for informado).

```
package org.acme;

import java.util.List;
import java.util.stream.Collectors;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.QueryParam;
import jakarta.ws.rs.core.MediaType;

@Path("/products")
public class ProductResource {

    private final List<Product> products = List.of(
        new Product(1L, "Notebook", "Eletronicos"),
        new Product(2L, "Mouse", "Eletronicos"),
        new Product(3L, "Cadeira", "Moveis"),
        new Product(4L, "Mesa", "Moveis"));

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Product> getProducts(@QueryParam("category") String category) {
        if (category == null) {
            return products;
        }
        return products.stream()
            .filter(p -> p.category().equalsIgnoreCase(category))
            .collect(Collectors.toList());
    }
}
```

🚨 **Dúvida comum:** `@QueryParam` lê da URL (`?category=x`), enquanto `@PathParam` lê de dentro do caminho (`/products/{id}`). Não confunda os dois — são mecanismos diferentes para capturar dados da requisição.

PASSO 3 — TESTAR OS DOIS CENÁRIOS

REQUISIÇÃO	O QUE ACONTECE INTERNAMENTE	RESULTADO
`GET /products`	`category` chega como `null` → cai no `if` → retorna a lista completa	4 produtos
`GET /products?category=Eletronicos`	`category` = "Eletronicos" → filtra o stream	Notebook e Mouse

```
curl http://localhost:8080/products
curl "http://localhost:8080/products?category=Eletronicos"
```

- [ ] Entendi a diferença entre @QueryParam e @PathParam
- [ ] Sei por que usar record em vez de classe com campos públicos
- [ ] Testei os dois endpoints e confirmei o filtro funcionando

🧠 AGORA TENTE VOCÊ

Adicione um segundo `@QueryParam` chamado `minPrice` (você vai precisar acrescentar um atributo `price` ao record `Product`) e filtre a lista para retornar apenas produtos com preço maior ou igual ao informado. Teste combinando os dois filtros ao mesmo tempo: `/products?category=Eletronicos&minPrice=1000`.

02 REST Client para consumir /users 🛠️

Tema: Rest Client — como um microsserviço fala com outro (ou com uma API externa).

Por que preciso disso? Em arquiteturas de microsserviços (e no exercício, ao consumir uma API pública de usuários), um serviço quase sempre precisa chamar outro pela rede. Fazer isso "na mão" com `HttpClient` significa escrever manualmente URL, headers, serialização e desserialização — repetitivo e propenso a erro.

O **MicroProfile Rest Client** resolve isso de um jeito elegante: você declara uma *interface Java* descrevendo os endpoints remotos (com as mesmas anotações JAX-RS que já usamos no Exercício 1) e o Quarkus **gera a implementação automaticamente** em tempo de build. Você injeta e usa como se fosse um serviço local.

PASSO 1 — DEPENDÊNCIAS (JÁ PRESENTES NO PROJETO)

```
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-rest-client</artifactId>
</dependency>
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-rest-client-jackson</artifactId>
</dependency>
```

PASSO 2 — CRIAR O DTO DE RESPOSTA

Assim como o `Product`, o `User` é só um "carregador de dados" vindo da API externa — outro candidato perfeito a `record`.

```
package org.acme;

public record User(Long id, String name, String username, String email) {
}
```

PASSO 3 — DECLARAR A INTERFACE DO CLIENTE

Seguindo a convenção da apostila (interfaces de Rest Client com prefixo `I`, como `IPayment` e `IBookCatalog`), criamos `IUserClient`. A anotação `@RegisterRestClient` diz ao Quarkus "gere uma implementação HTTP para esta interface". Repare que **não** colocamos `baseUri` na anotação — a URL virá de fora, no `application.properties` (Exercício 3), o que é a prática recomendada para não deixar endereços fixos no código.

```
package org.acme;

import java.util.List;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;

@RegisterRestClient
@Path("/users")
public interface IUserClient {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    List<User> listUsers();
}
```

PASSO 4 — INJETAR E USAR

A anotação `@RestClient` (junto com `@Inject`) diz ao CDI (o mecanismo de injeção de dependências do Quarkus) "me dê a implementação gerada para `IUserClient`". A partir daí, chamar `userClient.listUsers()` é indistinguível de chamar um método local — toda a complexidade de HTTP fica escondida.

```
package org.acme;

import java.util.List;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RestClient;

@Path("/local-users")
public class UserResource {

    @Inject
    @RestClient
    IUserClient userClient;

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<User> getUsers() {
        return userClient.listUsers();
    }
}
```

RESUMO DO PROCESSO (4 PASSOS)

PASSO	O QUE FAZ
1	Garantir as dependências rest-client no pom.xml
2	Criar o DTO (record) que representa a resposta remota
3	Declarar a interface com @RegisterRestClient + anotações JAX-RS
4	Injetar com @Inject @RestClient e configurar a URL fora do código

- ☐ Entendi que a interface só declara o contrato — o Quarkus implementa
- ☐ Sei por que a URL não fica hardcoded na anotação
- ☐ Testei /local-users e recebi a lista vinda da API externa

🧠 **AGORA TENTE VOCÊ**

Crie uma segunda interface, `IPostClient`, apontando para `/posts` na mesma API pública, com um método `getPost(@PathParam("id") Long id)`. Exponha um endpoint próprio, `GET /local-posts/{id}`, que injete essa interface e devolva o post correspondente.

03 Configuration: apontando o endpoint 🔧

Tema: Configuration — como o Quarkus lê propriedades e como isso se conecta ao Rest Client.

Onde ficam as configurações? O Quarkus (via MicroProfile Config) busca valores em várias fontes, na seguinte ordem de prioridade: parâmetros `-D` na linha de comando, variáveis de ambiente, arquivo `.env` na raiz do projeto e, por fim, `src/main/resources/application.properties`. Isso permite trocar um endpoint entre ambientes (dev/test/prod) sem recompilar o código.

Chave específica do Rest Client: o MicroProfile define um formato padronizado para configurar a URL base de uma interface sem precisar de anotação: `{pacote-completo.NomeDaInterface}/mp-rest/url`.

PASSO 1 — MONTAR A CHAVE DE CONFIGURAÇÃO

Como nossa interface é `org.acme.IUserClient`, a chave fica:

```
{pacote}.{Interface}/mp-rest/url = {url base}
org.acme.IUserClient/mp-rest/url = https://jsonplaceholder.typicode.com
```

PASSO 2 — ADICIONAR AO APPLICATION.PROPERTIES

```
# Configuracao do REST Client (interface IUserClient) - padrao MicroProfile
org.acme.IUserClient/mp-rest/url=https://jsonplaceholder.typicode.com
# Configuracao do REST Client (interface IPostClient) - padrao MicroProfile
org.acme.IPostClient/mp-rest/url=https://jsonplaceholder.typicode.com
```

🚨 **Atenção:** a URL configurada aqui é a *base*. O caminho final é composto com o `@Path("/users")` (ou `@Path("/posts")`) da interface, resultando em `https://jsonplaceholder.typicode.com/users` na hora da chamada real. Não aponte essa URL para `http://localhost:8080/...` — isso faria o client chamar a própria aplicação em vez da API pública, e a chamada falharia porque não existe nenhum recurso `/users` ou `/posts` exposto localmente.

PASSO 3 — SOBRESCREVER EM RUNTIME (SEM RECOMPILAR)

Qualquer propriedade pode ser sobrescrita na hora de subir a aplicação — útil para trocar de ambiente rapidamente:

```
./mvnw compile quarkus:dev -Dorg.acme.IUserClient/mp-rest/url=https://outra-api.com/v1
```

PASSO 4 — VARIAÇÃO POR PERFIL (DEV/TEST/PROD)

O Quarkus permite prefixar uma propriedade com `%perfil.` para que ela só valha naquele ambiente específico:

```
org.acme.IUserClient/mp-rest/url=https://jsonplaceholder.typicode.com
%prod.org.acme.IUserClient/mp-rest/url=https://api.producao.com/v1
```

- [ ] Entendi o formato {pacote.Interface}/mp-rest/url
- [ ] Sei sobrescrever uma propriedade via linha de comando
- [ ] Sei diferenciar valores por perfil com %prod./%dev.

🧠 **AGORA TENTE VOCÊ**

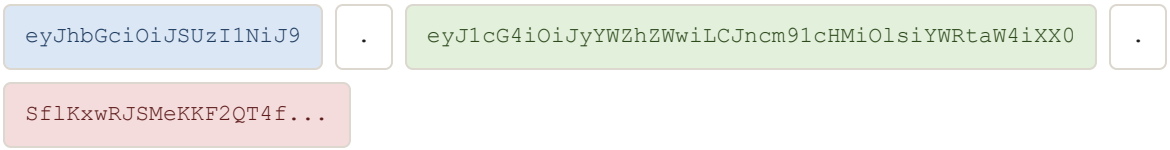
Adicione ao `application.properties` uma propriedade própria, `app.mensagem-boas-vindas`, injete-a com `@ConfigProperty` em um novo endpoint `GET /boas-vindas` e defina um valor diferente só para o perfil `%prod`. Rode em modo dev sobrescrevendo o valor via `-D` na linha de comando para confirmar a prioridade.

04 Restringindo acesso com JWT + @RolesAllowed 🔑

Tema: JSON Web Token — autenticação e autorização baseada em papéis (roles).

O que é um JWT? É um padrão (RFC 7519) para representar informações de forma compacta e assinada digitalmente, usado para controlar o acesso a serviços. Pense nele como a **pulseira de um festival**: depois que você mostra sua identidade uma vez na entrada (login), recebe uma pulseira (o token) que carrega, de forma verificável, quais áreas você pode acessar — sem precisar mostrar documento de novo em cada barraca.

Um JWT tem 3 partes separadas por ponto: **header** (algoritmo usado), **payload** (as informações/claims, como o usuário e suas roles) e **signature** (garante que ninguém alterou o conteúdo no caminho).



header (algoritmo) · payload (usuário + roles = "groups") · signature (integridade)

● PASSO 1 — ADICIONAR AS EXTENSÕES DE SEGURANÇA JWT

`quarkus-smallrye-jwt` valida tokens recebidos (necessário para `@RolesAllowed` funcionar); `quarkus-smallrye-jwt-build` permite *gerar* tokens dentro da própria aplicação — útil para o exercício, já que não temos um servidor de autenticação externo (como Keycloak).

```
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-smallrye-jwt</artifactId>
</dependency>
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-smallrye-jwt-build</artifactId>
</dependency>
```

● PASSO 2 — GERAR O PAR DE CHAVES RSA

A assinatura do token usa criptografia assimétrica: a **chave privada** assina (fica só no servidor que gera o token), a **chave pública** valida (pode ser distribuída). Geramos as duas com OpenSSL e colocamos em `src/main/resources`:

```
openssl genrsa -out privateKey.pem 2048
openssl rsa -in privateKey.pem -pubout -out publicKey.pem
```

PASSO 3 — CONFIGURAR ASSINATURA E VALIDAÇÃO

```
# Assinatura (usada por quem GERA o token — LoginResource)
smallrye.jwt.sign.key.location=privateKey.pem

# Validacao (usada por quem RECEBE o token — @RolesAllowed)
mp.jwt.verify.issuer=https://acme.org/issuer
mp.jwt.verify.publickey.location=publicKey.pem
```

PASSO 4 — ENDPOINT QUE GERA O TOKEN (LOGIN)

`@PermitAll` libera este endpoint sem exigir autenticação — faz sentido, já que é aqui que o usuário ainda "não tem" o token. O método usa a API fluente `Jwt.issuer(...)` para montar o token e inclui a role `admin` dentro de `.groups(...)` — é exatamente essa lista de "groups" que o Quarkus compara com o valor de `@RolesAllowed`.

```
package org.acme;

import java.util.HashSet;
import java.util.Set;
import jakarta.annotation.security.PermitAll;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import io.smallrye.jwt.build.Jwt;

@Path("/login")
public class LoginResource {

    private static final String ISSUER = "https://acme.org/issuer";

    @POST
    @PermitAll
    @Produces(MediaType.TEXT_PLAIN)
    public String generate(final String username) {
        Set<String> roles = new HashSet<>();
        roles.add("admin");

        return Jwt.issuer(ISSUER)
            .upn(username)
            .groups(roles)
            .sign();
    }
}
```

PASSO 5 — PROTEGER O ENDPOINT COM @ROLESALLOWED

Ao anotar o método com `@RolesAllowed("admin")`, o Quarkus intercepta a chamada *antes* de executar o corpo do método: extrai o JWT do header `Authorization: Bearer <token>`, valida a assinatura com a chave pública e confere se `"admin"` está entre os `groups` do token. Se algo falhar, a resposta é automática — não precisamos escrever nenhum `if`.

```
package org.acme;

import jakarta.annotation.security.RolesAllowed;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/admin")
public class AdminResource {

    @GET
    @RolesAllowed("admin")
    @Produces(MediaType.TEXT_PLAIN)
    public String adminOnly() {
        return "Acesso liberado: voce tem a role admin";
    }
}
```

PASSO 6 — TESTAR O FLUXO COMPLETO

CENÁRIO	COMANDO	RESULTADO ESPERADO
1. Gerar o token	`curl -X POST -d "rafael" localhost:8080/login`	200 OK + string do JWT
2. Chamar sem token	`curl localhost:8080/admin`	401 Unauthorized
3. Chamar com token	`curl -H "Authorization: Bearer $TOKEN" localhost:8080/admin`	200 OK

```
TOKEN=$(curl -s -X POST -d "rafael" http://localhost:8080/login)
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/admin
```

Anotações de segurança úteis:

ANOTAÇÃO	EFEITO
`@PermitAll`	Libera o acesso sem exigir token
`@RolesAllowed("admin")`	Exige token válido com a role/group informado
`@DenyAll`	Bloqueia o acesso para todo mundo
`@Inject JsonWebToken`	Permite ler claims do token dentro do método

🚨 **Para produção:** gerar tokens dentro da própria aplicação (como fizemos) é didático, mas em um cenário real o ideal é delegar autenticação a um provedor dedicado, como Keycloak, via extensão `quarkus-oidc` — o restante (`@RolesAllowed`) continua funcionando exatamente da mesma forma.

O QUE PODE DAR ERRADO

JWT costuma ser a parte onde alunos mais travam. Antes de "sair tentando", confira esta tabela:

SINTOMA	CAUSA PROVÁVEL	COMO RESOLVER
401 mesmo enviando o token	`Authorization` sem o prefixo `Bearer` (com espaço)	Enviar exatamente `Bearer <token>`, respeitando maiúscula e o espaço
401 logo depois de gerar o token	`mp.jwt.verify.issuer` diferente do `Jwt.issuer(...)` usado para assinar	Os dois valores precisam ser **idênticos** (mesma string)
403 Forbidden (não 401)	Token válido, mas sem a role certa dentro de `groups`	Conferir se `.groups(roles)` realmente inclui `"admin"`
Erro ao iniciar a aplicação citando "key location"	`privateKey.pem` / `publicKey.pem` não estão em `src/main/resources`	Gerar as chaves com OpenSSL e colocá-las nesse diretório
Token expira rápido demais nos testes	Duração padrão do claim `exp` é curta	Usar `.expiresIn(3600)` no builder do `Jwt.issuer(...)` durante o desenvolvimento

- [ ] Entendi as 3 partes de um JWT (header, payload, signature)
- [ ] Sei a diferença entre chave privada (assina) e pública (valida)
- [ ] Testei o fluxo completo: login → token → acesso liberado
- [ ] Testei o cenário de falha: sem token → 401

🧠 **AGORA TENTE VOCÊ**

Crie uma segunda role, `gerente`, e um novo endpoint `GET /relatorios` protegido com `@RolesAllowed({"admin", "gerente"})` (aceitando as duas roles). Depois gere um token só com `"gerente"` em `groups` e confirme que ele acessa `/relatorios` mas continua recebendo 403 em `/admin`.
